

Final Project Writeup: Semantic API Wrapper

Introduction:

Most applications communicate with each other using REST APIs. However, when a backend developer updates an endpoint, modifies a payload structure or updates a database schema, the frontend client breaks. While people aim to code in natural language, APIs still have an arbitrarily structured request/response format designed by people that is the opposite of natural. An attempt to solve this problem was the creation of MCP servers which allow people to call tools using natural language. However, this requires constantly contacting an MCP server on each request and also requires clients to run their own AI agents. As LLM calls become cheaper, I aim to create a semantic API wrapper which converts backend endpoints into a natural language API. By using LLM's as a proxy, developers can request data using natural language (e.g. "Get the 10 latest tasks"), and the LLM will map this request to the correct endpoints AND provide the correct type structures for the response and request. Furthermore, this can be run at build time rather than run time and act as a dev-tool for typescript, a common flavor of javascript for the frontend. Just like how VS code does type inference, it can do API call type inference too. This would require a typescript library for a client.

Design/Implementation

To simulate a real world scenario, I built a rest API written in Go which represents a regular backend. In this case, I created a simple task tracker app with basic CRUD routes that allows users to fetch tasks, update tasks, and create tasks. I also created some AUTH routes that use JSON Web Tokens which gives users a session without the server having to store any further data. I also created a simple [Next.js](#) app which called these API routes that I would then modify later using my frontend typescript library to use natural language.

The core of the project is the proxy server, which is written in Rust. This agent intercepts natural language queries from the frontend. During the setup phase, the agent accepts a file path to a backend. It reads the Go backend's routes and schemas. When a query is received, it uses the Claude API to map metadata of each backend route to a Claude tool which forces Claude to input the parameters needed to create an output that defines the response structure, the request structure, any query or path parameters and the shape of the error. Rust was selected for its fast-performance as well as memory safety to ensure that the proxy layer adds minimal latency as it will be used as a dev tool that may need to refresh on every file change.

The last aspect of the project is the typescript SDK and CLI in packages/client and packages/generate-types. Rather than querying the LLM at runtime which would introduce overhead on each API call, the CLI operates at build time. The developer is able to run the generate-types CLI which scans the frontend source code for natural language calls, pings the Rust agent to resolve them, and generates a run time cache, which hardcodes the natural language to rest endpoints, as well as generates typescript types for every single natural language call which forces the developer to ensure that their api usage adheres to the actual endpoint's specifications. Then, the developer just needs to replace their api calls with simple calls to the frontend library. The developer just needs to create a new agent, and they can talk to the agent to make API calls in natural language.

Discussion/Results:

The results were amazing. I could call my api with TaskAgent.call("Get the latest tasks"), and it would work perfectly as well as generate the proper types needed to require inputs like Authorization tokens for authenticated users. Using Claude to parse my backend routes and route my natural language queries resulted in perfectly structured results, likely due to the creation of a

Randy Sim

CS1680

Claude tool rather than extracting messages from a standard chat. I did build parts of this project with AI tools. AI was great at boilerplate heavy tasks such as creating a simple backend or a simple frontend. However, it struggled at creating an architecture such as a typescript library or a proxy layer to an API. I had to do a lot of tweaking and modifications myself as AI doesn't naturally come up with design ideas on its own. My project works as described because my frontend simply uses AI agent calls to interact with the API and all the functionality works, the network requests are exactly the same and there is no LLM latency on API calls.

Agent Analysis of Routes

```
[
  {
    "method": "POST",
    "path": "/auth/register",
    "semantic": "Register a new user account with email and password",
    "parameters": {
      "path": [],
      "query": [],
      "headers": [],
      "body": {
        "email": "string",
        "password": "string"
      }
    },
    "response": {
      "status": 201,
      "shape": {
        "email": "string",
        "id": "number"
      }
    },
    "errors": [
      {
        "status": 400,
        "message": "Invalid request body",
        "body": "string"
      },
      {
        "status": 400,
        "message": "Email and password are required",
        "body": "string"
      }
    ]
  }
]
```

CS1680

Agent Caching Natural Language Query to Route + Type

```

    error: null
  },
  "Get all tasks": {
    "method": "GET",
    "path": "/tasks",
    "semantic": "Retrieve all tasks belong to the user",
    "path_params": [],
    "query_params": [],
    "headers": [
      {
        "name": "Authorization",
        "type": "string",
        "required": true,
        "description": "Bearer JWT token for authentication",
        "default_value": null
      }
    ],
    "body_shape": null,
    "response_shape": {
      "tasks": [
        {
          "created_at": "string",
          "id": "number",
          "status": "string",
          "title": "string",
          "user_id": "number"
        }
      ]
    },
    "errors": [
      {
        "code": "400",
        "message": "Invalid request",
        "description": "The request is invalid due to missing or incorrect headers."
      },
      {
        "code": "401",
        "message": "Unauthorized",
        "description": "The user is not authenticated."
      },
      {
        "code": "404",
        "message": "Not found",
        "description": "The requested resource does not exist."
      }
    ]
  }
}

```

Conclusions/Future work:

Overall, this project exceeded my expectations. I remember using AI to do something similar in the past and it completely failed due to inconsistent outputs that required regeneration until it passed sanity checks. However, Anthropic's Claude proved that it could consistently generate formatted JSON output which made this entire idea trivial. If I could keep working on this project, I would replace the CLI tool with a VSCode extension that does the work of the CLI tool but after every save or file modification to make the experience perfectly seamless.